# 📓 PHP + MySQL (CRUD – Insert & Select)

## Professional Guide with Advanced Examples & Mini Project

---

## 📌 Class Objective

In this class, you will learn how to:

- Connect PHP with MySQL professionally
- Insert data securely into a database
- Retrieve and display data efficiently
- Apply real-world security practices
- Build a **mini advanced project** using professional structure

This class reflects **real-industry PHP development practices**.

---

## 🗂 Prerequisites

Before starting, make sure you have:

- XAMPP / WAMP / Laragon installed
- Basic PHP and HTML knowledge
- phpMyAdmin access
- Understanding of variables and forms

---

## 1️⃣ Understanding CRUD

CRUD represents the four fundamental database operations:

- **Create** → INSERT
- **Read** → SELECT
- **Update** → UPDATE
- **Delete** → DELETE

👉 This class focuses on **Create (Insert)** and **Read (Select)**, which are the foundation of all database-driven applications.

---

## 2 Database Design (Professional Approach)

### Create Database

```
CREATE DATABASE school;
USE school;
```

### Students Table (Industry-Level Design)

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    age INT NOT NULL,
    status ENUM('active','inactive') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

### Why This Design?

- `NOT NULL` → required fields
- `UNIQUE` → prevents duplicate emails
- `ENUM` → controlled status values
- `created_at` → audit & tracking

---

## 3 Database Connection (Reusable & Secure)

### db.php

```
<?php
define("DB_HOST", "localhost");
define("DB_USER", "root");
define("DB_PASS", "");
define("DB_NAME", "school");

$conn = mysqli_connect(DB_HOST, DB_USER, DB_PASS, DB_NAME);

if (!$conn) {
    die("Database connection failed");
}
?>
```

### Professional Rule

✔ Database connection **must never** be repeated in multiple files
✔ Always reuse `db.php`

---

## 4 INSERT Data (CREATE)

### HTML Form (add_student.php)

```
<form method="POST">
    <input type="text" name="name" placeholder="Student Name" required>
    <input type="email" name="email" placeholder="Email Address"
required>
    <input type="number" name="age" placeholder="Age" required>
    <button name="save">Add Student</button>
</form>
```

---

### ✗ Incorrect Way (For Learning)

```
INSERT INTO students VALUES ('$name','$email','$age');
```

⚠ Vulnerable to SQL Injection

---

### ☑ Correct Way (Prepared Statements)

```
<?php
require "db.php";

if (isset($_POST['save'])) {

    $name  = trim($_POST['name']);
    $email = trim($_POST['email']);
    $age   = trim($_POST['age']);

    if (empty($name) || empty($email) || empty($age)) {
        echo "All fields are required";
        exit;
    }

    $stmt = mysqli_prepare(
        $conn,
        "INSERT INTO students (name, email, age) VALUES (?, ?, ?)"
    );

    mysqli_stmt_bind_param($stmt, "ssi", $name, $email, $age);
    mysqli_stmt_execute($stmt);

    echo "Student successfully added";
}
?>
```

### Professional Notes

✔ Prevents SQL Injection
✔ Handles validation
✔ Clean and readable code

## 5 SELECT Data (READ)

### Fetch All Records

```
$sql = "SELECT * FROM students ORDER BY id DESC";
$result = mysqli_query($conn, $sql);
```

### Display in Table (Secure Output)

```
<table border="1" cellpadding="10">
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Age</th>
    <th>Status</th>
    <th>Created</th>
</tr>

<?php while ($row = mysqli_fetch_assoc($result)) { ?>
<tr>
    <td><?= $row['id']; ?></td>
    <td><?= htmlspecialchars($row['name']); ?></td>
    <td><?= htmlspecialchars($row['email']); ?></td>
    <td><?= $row['age']; ?></td>
    <td><?= $row['status']; ?></td>
    <td><?= $row['created_at']; ?></td>
</tr>
<?php } ?>
</table>
```

## 6 Advanced SELECT Examples

### SELECT with Condition

```
SELECT * FROM students WHERE status = 'active';
```

### Search Feature

```
$keyword = $_GET['search'] ?? '';

$sql = "SELECT * FROM students
        WHERE name LIKE '%$keyword%'
        OR email LIKE '%$keyword%'";
```

### Pagination (Professional Concept)

```
$limit = 5;
$page = 1;
$offset = ($page - 1) * $limit;

SELECT * FROM students LIMIT $offset, $limit;
```

## 7 MINI ADVANCED PROJECT

## Student Management System (Mini – Professional)

### Project Folder Structure

```
student_management/
├── config/
│   └── db.php
├── public/
│   ├── index.php
│   ├── add_student.php
│   ├── view_students.php
├── assets/
│   └── style.css
```

## 1) Database Setup (phpMyAdmin / MySQL)

### (A) Create DB

```
CREATE DATABASE school;
USE school;
```

### (B) Create Table

```
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  age INT NOT NULL,
  status ENUM('active','inactive') DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

## 2) config/db.php

**File:** student_management/config/db.php

```
<?php
// config/db.php

define("DB_HOST", "localhost");
define("DB_USER", "root");
define("DB_PASS", "");
define("DB_NAME", "school");

$conn = mysqli_connect(DB_HOST, DB_USER, DB_PASS, DB_NAME);

if (!$conn) {
    die("Database connection failed: " . mysqli_connect_error());
}
```

---

### 3) `assets/style.css`

**File:** `student_management/assets/style.css`

```
/* assets/style.css */
body { font-family: Arial, sans-serif; margin: 30px; }
nav a { margin-right: 12px; text-decoration: none; }
.container { max-width: 900px; margin: auto; }
.card { border: 1px solid #ddd; padding: 16px; border-radius: 8px;
margin-top: 16px; }
input, button, select { padding: 10px; margin: 6px 0; width: 100%; }
table { width: 100%; border-collapse: collapse; margin-top: 12px; }
th, td { border: 1px solid #ddd; padding: 10px; }
th { background: #f5f5f5; text-align: left; }
.success { color: green; }
.error { color: red; }
.inline { display: flex; gap: 10px; }
.inline > * { flex: 1; }
.pagination a { margin-right: 8px; }
.small { font-size: 0.9rem; color: #555; }
```

---

### 4) `public/index.php` (Dashboard)

**File:** `student_management/public/index.php`

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management</title>
  <link rel="stylesheet" href="../assets/style.css">
</head>
<body>
<div class="container">
  <h2>Student Management System</h2>
  <nav>
    <a href="add_student.php">Add Student</a>
    <a href="view_students.php">View Students</a>
  </nav>

  <div class="card">
    <p class="small">
      This mini project demonstrates professional INSERT + SELECT with:
      Prepared Statements, Validation, XSS Protection, Search,
Pagination.
    </p>
  </div>
</div>
</body>
</html>
```

---

## 5) `public/add_student.php` (INSERT + Validation + Prepared Statement)

**File:** `student_management/public/add_student.php`

```
<?php
require "../config/db.php";

$message = "";
$messageClass = "";

if (isset($_POST['save'])) {

    $name  = trim($_POST['name'] ?? "");
    $email = trim($_POST['email'] ?? "");
    $age   = trim($_POST['age'] ?? "");
    $status = trim($_POST['status'] ?? "active");

    // Basic validation
    if ($name === "" || $email === "" || $age === "") {
        $message = "All fields are required.";
        $messageClass = "error";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $message = "Invalid email format.";
        $messageClass = "error";
    } elseif (!ctype_digit($age) || (int)$age < 1 || (int)$age > 120) {
        $message = "Age must be a valid number (1-120).";
        $messageClass = "error";
    } elseif ($status !== "active" && $status !== "inactive") {
        $message = "Invalid status selected.";
        $messageClass = "error";
    } else {

        // Prepared statement (secure insert)
        $stmt = mysqli_prepare(
            $conn,
            "INSERT INTO students (name, email, age, status) VALUES (?,
?, ?, ?)"
        );

        mysqli_stmt_bind_param($stmt, "ssis", $name, $email, $age,
$status);

        if (mysqli_stmt_execute($stmt)) {
            $message = "Student added successfully!";
            $messageClass = "success";
        } else {
            // Duplicate email error example
            if (mysqli_errno($conn) === 1062) {
                $message = "This email is already registered
(duplicate).";
            } else {
                $message = "Insert failed: " . mysqli_error($conn);
            }
            $messageClass = "error";
        }
```

```
        mysqli_stmt_close($stmt);
    }
}
?>
<!DOCTYPE html>
<html>
<head>
  <title>Add Student</title>
  <link rel="stylesheet" href="../assets/style.css">
</head>
<body>
<div class="container">
  <h2>Add Student</h2>
  <nav>
    <a href="index.php">Dashboard</a>
    <a href="view_students.php">View Students</a>
  </nav>

  <div class="card">
    <?php if ($message !== ""): ?>
      <p class="<?= $messageClass; ?>"><?= htmlspecialchars($message);
?></p>
    <?php endif; ?>

    <form method="POST">
      <label>Name</label>
      <input type="text" name="name" placeholder="Student Name"
required>

      <label>Email</label>
      <input type="email" name="email" placeholder="Email Address"
required>

      <label>Age</label>
      <input type="number" name="age" placeholder="Age" required>

      <label>Status</label>
      <select name="status">
        <option value="active">active</option>
        <option value="inactive">inactive</option>
      </select>

      <button name="save">Add Student</button>
    </form>
  </div>
</div>
</body>
</html>
```

---

## 6) `public/view_students.php` (SELECT + Search + Pagination + XSS Safe Output)

**File:** `student_management/public/view_students.php`

```php
<?php
require "../config/db.php";

// Search
$search = trim($_GET['search'] ?? "");

// Pagination
$page  = (int)($_GET['page'] ?? 1);
$limit = 5;
if ($page < 1) $page = 1;
$offset = ($page - 1) * $limit;

// Build WHERE clause (simple, readable)
$whereSql = "";
$params = [];
$types = "";

if ($search !== "") {
    // We'll use prepared statements even for search (professional)
    $whereSql = "WHERE name LIKE ? OR email LIKE ?";
    $like = "%" . $search . "%";
    $params = [$like, $like];
    $types = "ss";
}

// Count total rows (for pagination)
if ($whereSql === "") {
    $countQuery = "SELECT COUNT(*) AS total FROM students";
    $countResult = mysqli_query($conn, $countQuery);
    $total = (int)mysqli_fetch_assoc($countResult)['total'];
} else {
    $countStmt = mysqli_prepare($conn, "SELECT COUNT(*) AS total FROM
students $whereSql");
    mysqli_stmt_bind_param($countStmt, $types, ...$params);
    mysqli_stmt_execute($countStmt);
    $countRes = mysqli_stmt_get_result($countStmt);
    $total = (int)mysqli_fetch_assoc($countRes)['total'];
    mysqli_stmt_close($countStmt);
}

$totalPages = (int)ceil($total / $limit);

// Fetch rows
if ($whereSql === "") {
    $stmt = mysqli_prepare($conn, "SELECT * FROM students ORDER BY id
DESC LIMIT ? OFFSET ?");
    mysqli_stmt_bind_param($stmt, "ii", $limit, $offset);
} else {
    $stmt = mysqli_prepare($conn, "SELECT * FROM students $whereSql
ORDER BY id DESC LIMIT ? OFFSET ?");
    // merge params + limit + offset
    $params2 = array_merge($params, [$limit, $offset]);
    mysqli_stmt_bind_param($stmt, $types . "ii", ...$params2);
}

mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
```

```
?>
<!DOCTYPE html>
<html>
<head>
  <title>View Students</title>
  <link rel="stylesheet" href="../assets/style.css">
</head>
<body>
<div class="container">
  <h2>Students</h2>
  <nav>
    <a href="index.php">Dashboard</a>
    <a href="add_student.php">Add Student</a>
  </nav>

  <div class="card">
    <form method="GET" class="inline">
      <input type="text" name="search" placeholder="Search by name or
email..." value="<?= htmlspecialchars($search); ?>">
      <button>Search</button>
    </form>

    <p class="small">Total Students Found: <?= $total; ?></p>

    <table>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Age</th>
        <th>Status</th>
        <th>Created</th>
      </tr>

      <?php while ($row = mysqli_fetch_assoc($result)) { ?>
        <tr>
          <td><?= $row['id']; ?></td>
          <td><?= htmlspecialchars($row['name']); ?></td>
          <td><?= htmlspecialchars($row['email']); ?></td>
          <td><?= (int)$row['age']; ?></td>
          <td><?= htmlspecialchars($row['status']); ?></td>
          <td><?= htmlspecialchars($row['created_at']); ?></td>
        </tr>
      <?php } ?>
    </table>

    <div class="pagination" style="margin-top: 12px;">
      <?php
      // Pagination links (keep search keyword)
      for ($p = 1; $p <= max(1, $totalPages); $p++) {
          $query = http_build_query(["search" => $search, "page" =>
$p]);
          echo '<a href="view_students.php?' . $query . '">' . $p .
'</a>';
      }
      ?>
    </div>
```

```
  </div>
</div>
</body>
</html>
<?php
mysqli_stmt_close($stmt);
?>
```

### Project Features

- Secure student insertion
- Student listing
- Search by name/email
- Status control (active/inactive)
- Pagination ready
- Professional structure

---

### Business Rules (Important)

- Duplicate emails not allowed
- Age must be numeric
- Only active students shown by default
- Secure database operations only

---

## Professional Security Rules

✔ Always use prepared statements
✔ Escape output (`htmlspecialchars`)
✔ Validate inputs
✔ Never trust user data
✔ Separate config, logic, and UI

---

## Professional Development Tips

- Write clean, readable code
- Use meaningful variable names
- Keep SQL readable
- Follow folder structure
- Comment complex logic
- Think like a system designer, not just a coder